Name: **CSE 450, Fall 2026 – Quiz 1A**

All questions below assume lexer-style regular expression syntax as found in Emplex or flex (and as taught in class), including the implicit rule that unknown matches return the next single character by itself. Each question is worth 1 point, for a total of 10 points.

For the first three questions, consider a lexer with the following token types:

PRINT	`"print"`
KEYWORD	`[a-z]+`
NUM	`[0-9]+`
VAR	`[a-zA-Z_][a-zA-Z0-9_]*`

1. What would the lexer return for the input: `print2`

A. A single PRINT token
B. A single KEYWORD token
C. A single NUM token
D. A single VAR token
E. A PRINT and then a NUM token
F. A KEYWORD and then a NUM token

2. Given the same lexer as the previous question, what would be returned for the input: -42

A. A single PRINT token
B. A single KEYWORD token
C. A single NUM token
D. A single VAR token
E. The character '-' and then a NUM token
F. A KEYWORD and then a NUM token

3. Given an input sequence of letters (lowercase, capital, or a mix) other than "`print`", when will KEYWORD be returned as compared to VAR?

A. KEYWORD will always have precedence, so VAR will never be returned.

B. VAR will always have precedence, so KEYWORD will never be returned.

C. KEYWORD comes first, so it will always be given precedence for sequences beginning with a lowercase letter. VAR will be returned only if the sequence begins with a capital letter.

D. KEYWORD comes first, so it will always be given precedence for sequences consisting only of lowercase letters. VAR will be returned if the sequence has any capital letters in it.

E. VAR comes last, so it will always be given precedence for sequences consisting of multiple letters; KEYWORD will be used only for a single lowercase letter.

4. How many words are in the regular language represented by ℒ(`[ab][cd][ef]`) ?

A. 4 words
B. 6 words
C. 8 words
D. 10 words
E. 12 words
F. 16 words

5. Which of the following regular expressions will match positive integers with exactly one '9' in them? For example, it should match 9, 911, or 789, but it should reject 1, 20, and 345 (for not having any 9's) as well as 9009, 99, and 9192939 (for having too many 9s).

A. `[0-9]+9[0-9]+`

B. `[0-8]+9[0-8]+`

C. `[0-8]*9[0-8]*`

D. `[0-9]*9[0-9]*`

E. `[0-8]9?[0-8]`

F. `[0-9]9?[0-9]`

6. You want to match the literal string `C++` and nothing else. Using that character sequence directly in a regular expression will not work, because + means "one or more of the preceding item", so `C++` would match `C`, `CC` and `CCCCC`, not what you want. There are multiple techniques to take the + literally. Which of the following will NOT work?

A. `C[+]{2}`

B. `C"++"`

C. `C\++`

D. `"C++"`

E. `C[+][+]`

F. `C\+\+`

7. How do the following regular expressions compare to each other?

A: `(ab)+` **B: `a(ba)*b`**

***Hint*: Consider the strings "ab", "aba", "abab", and "abb".**

A. A matches a subset of the sequences that B matches.

B. B matches a subset of the sequences that A matches.

C. These two regular expressions are identical in what they match and don't match.

D. These two regular expressions match completely distinct sequences.

E. None of the above; each regular expression matches some distinct sequences and some of the same sequences.

8. You need a regular expression that matches sequences of letters. The following pattern correctly matches `ABC`, `xyz`, and `Hello`, but will it work in all cases?: `[a-z|A-Z]+`

A. Yes. It will always correctly match all sequences of letters and nothing else.

B. No. The + allows the class to be re-entered, but will not allow the same character to be used more than once, so `AAA` or `xx` would not be allowed.

C. No. The `|` separates the brackets into two independent patterns, `[a-z]` and `[A-Z]`, and so the + applies only to the second.

D. No. Inside a character class, `|` loses its special meaning and stands for the literal vertical-bar character, so the character class means “a lowercase letter, a vertical bar, or an uppercase letter.”

E. No. Ranges in a character class must be in ASCII order, so `A-Z` must come before `a-z`.

F. No. A character class may contain only one range; everything after the first range is ignored, so the pattern reduces to `[a-z]+`.

9. Given the regular expression `([a-z][a-z][a-z])+`, which of the following descriptions is most accurate?

A. All sequences of lowercase letters whose length is a positive multiple of three. (E.g., "cat", "catnip", and "dogcatnip", but not "ox", "cats", or "elephant".)

B. All possible triples of lowercase letters repeated together any number of times. (E.g., "abcabc" and "xyzxyzxyz", but not "abcdef".)

C. All sequences of lowercase letters of any length. (E.g., "a", "cat", and "cats", but not "42".)

D. All sequences of lowercase letters of length three or greater. (E.g., "cat", "cats", and "elephant", but not "ox".)

E. All sequences of exactly three lowercase letters. (E.g., "cat" and "dog", but not "catnip".)

10. Which of the following regular expressions will match all positive integers that don't have leading zeros? For example, it should match 2020, 420, 404, or 9, but reject 007.

A. `[1-9][0-9]*`

B. `[1-9]+[0-9]+`

C. `[1-9]+0*[1-9]*`

D. `[1-9](0[1-9])*`

E. `[1-9]+[0-9]?`

EXTRA CREDIT:

You need a regular expression that matches times on a 24-hour clock, written as exactly two digits, a colon, and two more digits. It should match 00:00, 09:15, 13:45, and 23:59, but reject 24:00 and 23:60 (out of range), 9:15 (missing the leading zero), and 99:99.

Write a regular expression that will work in this scenario.